

Name of the Student	MADDIRALA BALAMURALI KRISHNA
Reg. No	192325015
GitHub Link	https://github.com/krishnalsk/MLA0403-DEEP-LEARNING.git

SIMATS ENGINEERING
CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning

COURSE CODE: MLA0403

COURSE OUTCOME COVERED

CO 4: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. BL-6

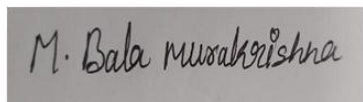
Course Outcome Weightage: 10% Duration: 90 minutes

Assessment Tool Weightage: 40% Mark: 25

Code of Conduct

I *MADDIRALA BALAMURALI KRISHNA(1923250151)* (certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



MADDIRALA BALAMURALI KRISHNA

Signature of the student:

Name of the student:

Criteria	Understanding of Industry Problem (2)	Application of Engineering Concepts (2.5)	Solution Design (2.5)	Use of Tools (2)	Analysis (1)	Total (10)
Weightage	20%	25%	25%	20%	10%	100%
Q1						
Q2						
Q3						
Q4						
Q5						
Total						

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Answer 1: Transfer Learning for Dermoscopic Skin Cancer Classification**1.1 Understanding the Industry Problem**

A healthcare startup needs to classify dermoscopic images (e.g., melanoma vs benign lesions, or multi-class per HAM10000/ISIC taxonomy) with a limited labeled dataset. Constraints: data scarcity, high cost of misclassification (false negatives are clinically dangerous), and need for model explainability for regulatory/clinical trust.

1.2 Model Choice & Justification

- Backbone: EfficientNet-B0/B3 or ResNet-50 pre-trained on ImageNet. EfficientNet is preferred in current practice for medical imaging startups because its compound scaling gives strong accuracy-per-parameter, which matters when the labeled set is small and inference may run on limited clinical hardware.
- Both architectures have been extensively validated on dermoscopic benchmarks (ISIC 2019/2020, HAM10000) in published literature, which reduces deployment risk versus a custom architecture trained from scratch on a small proprietary set.

1.3 Freeze vs Fine-tune Strategy

- Freeze: Early stem and first 2 blocks — these capture generic edges/color-gradient/texture filters transferable from natural images to skin images.
- Fine-tune: Final 2–3 blocks (deeper semantic layers) at a low learning rate, since lesion-specific patterns (asymmetry, border irregularity, color variegation — the clinical ABCDE features) differ from ImageNet object semantics.
- Replace: Final classifier — GlobalAveragePooling → Dense(128, ReLU) → Dropout(0.5) → Dense(num_classes, Softmax).

1.4 How This Improves Classification Performance

- Reduces overfitting risk on the small labeled set by anchoring most weights to features already learned from millions of ImageNet images.
- Two-phase training (train head first at LR 1e-3, then fine-tune unfrozen blocks at LR 1e-5) prevents the randomly initialized head from corrupting useful pre-trained filters early on.
- Test-time augmentation and heavy training-time augmentation (rotation, flips, color jitter — dermoscopic images have no fixed orientation) further boosts generalization given limited data.
- Grad-CAM visualization is applied post-hoc so clinicians can verify the model is attending to the lesion, not artifacts (rulers, hair, skin markers) — critical for regulatory acceptance.

1.5 Tools & Evaluation

- Tools: TensorFlow/Keras (EfficientNetB0 in tf.keras.applications) or PyTorch (timm library); ISIC Archive / HAM10000 for pretraining-adjacent benchmarking.
- Evaluation: Sensitivity (recall for malignant class is the priority metric), Specificity, AUC-ROC, and calibration analysis — a startup deploying a clinical-adjacent tool must report confidence, not just point predictions.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Answer 2: DenseNet + PixelNet Architecture for Pixel-Level Road-Scene Analysis**2.1 Understanding the Industry Problem**

A smart-city road-scene system needs dense, pixel-accurate labeling (semantic segmentation) of roads, pedestrians, vehicles, and traffic signs — not just bounding boxes — to support downstream tasks like free-space estimation and pedestrian-safety alerts. Constraints: real-time or near-real-time inference on edge/roadside hardware, and robustness to class imbalance (road pixels vastly outnumber pedestrian/sign pixels).

2.2 Proposed Architecture

- Backbone (feature extractor): DenseNet-121, pre-trained on ImageNet, used as the encoder. Its dense blocks are retained up through the final block, discarding the ImageNet classification head.
- Pixel-level prediction head (PixelNet-style): Rather than a heavy symmetric decoder, follow the PixelNet hypercolumn approach — for every pixel, concatenate the feature-map activations from multiple DenseNet layers (early, middle, late — each bilinearly upsampled to the input resolution) into a single 'hypercolumn' descriptor that captures both fine spatial detail (early layers) and high-level semantics (late layers).
- Classifier: A shared multi-layer perceptron (implemented as a stack of 1×1 convolutions) is applied to each pixel's hypercolumn to output a softmax over 5 classes (road, pedestrian, vehicle, traffic sign, background).
- Practical note: during training, PixelNet's efficiency trick — sparsely sampling a subset of pixels per image per iteration rather than computing loss densely over every pixel — cuts training compute substantially with negligible accuracy loss; at inference, prediction is computed for all pixels.

2.3 How This Improves Feature Reuse, Accuracy, and Efficiency

- Feature reuse: DenseNet's dense connectivity means every layer accesses all preceding feature maps directly, so low-level edge/lane-marking features remain available to the pixel classifier without being re-learned deeper in the network — directly benefiting fine-grained boundaries (curb lines, sign edges).
- Segmentation accuracy: the hypercolumn combination lets the classifier jointly use precise localization (from shallow, high-resolution layers) and strong semantic context (from deep layers), which is exactly the localization/semantics trade-off that pixel-level tasks need.
- Computational efficiency: DenseNet needs fewer parameters than a comparably deep ResNet/VGG for the same representational power (feature concatenation avoids redundant re-computation), and PixelNet's per-pixel MLP avoids the parameter/computation overhead of a full transposed-convolution decoder (e.g., as in U-Net or SegNet).

2.4 Tools & Real-World Alternatives

- Tools: PyTorch/TensorFlow with DenseNet-121 pretrained weights, custom 1×1-conv pixel head; datasets — Cityscapes or Mapillary Vistas are the standard real-world benchmarks for exactly this road-scene task.
- A closely related, widely deployed real-world design is FC-DenseNet ('The One Hundred Layers Tiramisu'), which fully replaces the PixelNet MLP head with a DenseNet-based upsampling decoder — worth citing as an alternative if a fully convolutional decoder is preferred over the hypercolumn/MLP approach.
- Evaluation: mean Intersection-over-Union (mIoU) per class, with particular attention to pedestrian/traffic-sign IoU since these minority classes matter most for safety despite their small pixel count.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Answer 3: Bidirectional RNN Encoder–Decoder for a Context-Preserving Multilingual Chatbot

3.1 Understanding the Industry Problem

The chatbot must translate customer queries across languages while preserving contextual meaning — not just word-for-word substitution — since customer intent (complaint, request, urgency) must survive translation for downstream response generation. Constraint: variable-length input/output, and meaning that depends on both preceding and following words within a query.

3.2 Architecture

- Encoder: A Bidirectional LSTM (or GRU) processes the tokenized input query in both directions. The forward LSTM reads left-to-right, the backward LSTM reads right-to-left; their hidden states at each timestep are concatenated, giving every token a representation informed by full-sentence context.
- Attention layer: Bahdanau-style additive attention computes, at each decoder step, a weighted combination over all encoder hidden states (not just a single final context vector), letting the decoder focus on the most relevant source words when generating each target word.
- Decoder: A unidirectional LSTM generates the translated query token-by-token, conditioned on the attention context and the previously generated token (teacher forcing during training).
- Output layer: Dense + Softmax over the target-language vocabulary at each step; beam search at inference for higher-quality output over greedy decoding.

3.3 Why Bidirectional Processing Improves Translation Accuracy

- Many source-language cues that determine correct translation (negation scope, modifiers, idioms) depend on words that appear after the ambiguous token — a unidirectional encoder cannot see this at the time it encodes that token, while the backward pass supplies it.
- Concatenated forward+backward states give the attention mechanism richer, disambiguated source representations to select from, which measurably improves translation of syntactically complex or reordered sentences — a common case across customer queries in different languages.
- For a chatbot specifically, preserving intent-bearing context (e.g., "not", "urgent", "cancel") depends on this two-directional context capture; unidirectional-only models are more prone to dropping such cues in longer queries.

3.4 Tools & Evaluation

- Tools: TensorFlow/Keras or PyTorch Seq2Seq with Bidirectional-LSTM encoder + attention; in current industry practice this task would typically use a pretrained multilingual Transformer (e.g., mBART-50, NLLB-200, or a fine-tuned M2M-100) rather than a from-scratch RNN, given superior quality and parallel training — worth noting as the practical production recommendation, with the BiRNN design serving as the foundational architecture requested.
- Evaluation: BLEU/chrF for translation quality, plus intent-preservation accuracy (does the translated query trigger the same downstream chatbot intent classification as the original).

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Answer 4: LSTM-Based Stock Price Forecasting with BPTT Training

4.1 Understanding the Industry Problem

Stock prices form a noisy, non-stationary time series where the relevant history can span from a few days to several weeks (momentum, earnings cycles).

The model must learn temporal dependencies of varying length while remaining trainable via gradient descent — the core motivation for using LSTM over a plain RNN.

4.2 Model Design

- Input: sliding window of the last N days (e.g., N=30–60) of features — Open, High, Low, Close, Volume, plus engineered indicators (moving averages, RSI) — normalized (Min-Max or returns-based, not raw price, to keep the series closer to stationary).
- Architecture: Input(N, features) → LSTM(128, return_sequences=True, dropout=0.2) → LSTM(64) → Dense(32, ReLU) → Dense(1) predicting next-day price (or return).
- For multi-step forecasting, either an autoregressive loop or a Seq2Seq (encoder-decoder LSTM) variant is used, depending on the required forecast horizon.

4.3 How BPTT Trains the Network

- The LSTM is conceptually 'unrolled' across all N timesteps into an equivalent feed-forward computation graph, with shared weights at every timestep.
- Forward pass: at each timestep t, the cell computes the forget/input/output gate activations and updates cell state C_t and hidden state h_t from C_{t-1} , h_{t-1} , and the input x_t , finally producing the prediction and loss at the output step(s).
- Backward pass: the gradient of the loss is propagated backward through this unrolled graph, from the final timestep back to $t=1$, using the chain rule at each timestep — this is Backpropagation Through Time. Gradients with respect to shared weights are summed across all timesteps before the weight update.
- Because full BPTT over long windows is expensive and can still suffer instability, Truncated BPTT is used in practice — gradients are backpropagated only a fixed number of steps (e.g., 20–30) rather than the full history, with the hidden/cell state carried forward across truncation boundaries.
- Gradient clipping (norm $\approx 1-5$) is applied since even LSTM gradients can occasionally explode over long unrolled sequences.

4.4 Why LSTM Over a Traditional RNN

- A vanilla RNN's gradient during BPTT is a repeated product of the same weight matrix and derivative term across timesteps, which vanishes (or explodes) exponentially with sequence length — in practice this means a plain RNN effectively 'forgets' dependencies beyond ~ 10 timesteps, too short to capture weekly/monthly market patterns.
- LSTM's cell state acts as an additive, largely linear 'error carousel' gated by the forget/input/output gates, so gradients can flow across many more timesteps without vanishing — directly enabling the network to learn longer-range dependencies (e.g., a multi-week trend) relevant to stock forecasting.
- The forget gate specifically lets the network learn to discard stale information (e.g., pre-earnings-report price levels) while the input gate lets it selectively incorporate new signal — behavior a plain RNN cannot represent.

4.5 Tools & Evaluation

- Tools: TensorFlow/Keras or PyTorch LSTM; yfinance/Alpha Vantage for historical OHLCV data acquisition.
- Evaluation: RMSE/MAE on held-out (chronologically split, never randomly shuffled) test data, directional accuracy (correct up/down movement prediction), and a naive-baseline (previous-day-price) comparison — a documented and important sanity check since financial time series are notoriously close to a random walk.
- Limitation: no LSTM design overcomes the fundamentally low signal-to-noise ratio of market data; results should be reported with realistic expectations and backtested with transaction-cost-aware simulation before any trading use.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Answer 5: Integrated Transfer Learning + LSTM Encoder–Decoder for Automatic Video Captioning

5.1 Understanding the Industry Problem

The streaming company needs natural-language captions generated automatically from uploaded video for accessibility. This requires combining two sub-problems: (1) extracting meaningful visual features from a temporal sequence of video frames, and (2) generating a coherent, variable-length natural-language sentence describing those features — a visual-to-text sequence generation task.

5.2 Integrated Architecture

- Component 1 — Transfer Learning (Visual Feature Extraction): A CNN pre-trained on ImageNet (InceptionV3 or ResNet-50) is applied to sampled frames (e.g., 1 frame/second) to extract a fixed-length feature vector per frame from the penultimate (pre-classification) layer; the classification head is discarded. Convolutional layers are frozen (or only the last block fine-tuned) since general visual-object recognition transfers well and full retraining is unnecessary and data-inefficient for this sub-task.
- Component 2 — LSTM Encoder: The sequence of per-frame CNN feature vectors is fed into an encoder LSTM (or Bidirectional LSTM) that aggregates them into a temporal video representation, capturing motion/event context across frames (this is the classic S2VT / 'Sequence to Sequence – Video to Text' design pattern used in production video-captioning research).
- Component 3 — LSTM Decoder with Attention: A decoder LSTM generates the caption word-by-word, at each step attending over the encoder's frame-level hidden states (temporal attention) so it can focus on the most relevant moments of the video for the word currently being generated — e.g., attending to the frame where an action occurs.
- Output layer: Dense + Softmax over the caption vocabulary at each decoding step; beam search at inference.

5.3 Role of Each Component

- Transfer-learned CNN: converts raw pixels into a compact, semantically meaningful visual descriptor per frame — without this, the LSTM encoder would have to learn visual understanding from scratch on a captioning dataset far too small for that purpose.
- Encoder LSTM: compresses the variable-length frame-feature sequence into a temporal context, modeling how the scene changes/evolves across the video, not just what is present in a single frame.
- Decoder LSTM + attention: converts the visual/temporal context into fluent, grammatical natural language, one token at a time, dynamically re-weighting which frames matter for each generated word.

5.4 Why This Produces Accurate Captions

- Transfer learning solves the data-scarcity problem for the visual side — captioning-labeled video datasets are far smaller than ImageNet, so reusing pretrained visual features is what makes the overall system trainable at all.
- The encoder-decoder split cleanly separates 'understanding the video' from 'generating language', letting each be optimized with appropriate architectures and loss functions (cross-entropy over vocabulary for the decoder).
- Temporal attention prevents the classic S2VT weakness of compressing an entire video into one fixed vector, which loses fine-grained event ordering — attention preserves the ability to reference specific moments.

5.5 Tools & Evaluation

- Tools: TensorFlow/Keras or PyTorch; pretrained InceptionV3/ResNet50 (or a video-native backbone like I3D/VideoMAE for stronger motion features); MSVD and MSR-VTT are the standard real-world benchmark datasets for this exact task.
- Evaluation: BLEU-4, METEOR, and CIDEr (the standard captioning metrics, CIDEr in particular correlates best with human judgment for this task), plus manual accessibility review since captions serve a real accessibility function, not just a benchmark score.